MVLU COLLEGE
WSN PRACTICAL 2

Aim:Understanding TinyOS Architecture and Execution Model.
Code:

```
print("Hariprasaad Vishwakarma T127")

import time
import queue
import threading


class TinyOSSchedulerSimulation:

    def __init__(self):
        # The core bounded FIFO task queue used by the TinyOS scheduler loop
        self.task_queue = queue.Queue(maxsize=8)
        self.is_running = True
        self.system_lock = threading.Lock()

    def post_task(self, task_name, computation_time):
        """Simulates the nesC 'post taskName()' command."""
        try:
            self.task_queue.put_nowait((task_name, computation_time))
            print(
                f"[SYSTEM]: Task '{task_name}' successfully posted "
                f"to FIFO queue."
            )
        except queue.Full:
            print(
                f"[WARNING]: FIFO Task Queue Full! "
                f"Task '{task_name}' dropped."
            )

    def trigger_hardware_event(self, event_name, bound_task_name, comp_time):
        """Simulates an asynchronous hardware interrupt signaling an event."""

        print(f"\n⚡ [HARDWARE INTERRUPT]: Event '{event_name}' fired!")

        # In TinyOS, an event handler responds instantly
        # and posts a deferred task.
        self.post_task(bound_task_name, comp_time)

    def run_scheduler_loop(self):
        """The main TinyOS execution loop."""

        print(
```

Hariprasad T127

```
    "[INIT]: TinyOS Scheduler Loop Initialized. "
    "Waiting for tasks..."
)

while self.is_running or not self.task_queue.empty():

    try:
        # Emulate entering low-power sleep mode if queue is empty
        task_name, comp_time = self.task_queue.get(timeout=1)

        print(
            f"\n🔄 [SCHEDULER]: Dispatching '{task_name}' "
            f"(FIFO - Non-preemptive execution)"
        )

        # Non-preemptive execution block
        with self.system_lock:

            start_time = time.time()

            # Simulate processing computational workload
            time.sleep(comp_time)

            elapsed_time = time.time() - start_time

            print(
                f"✅ [TASK COMPLETE]: '{task_name}' ran to "
                f"completion in {elapsed_time:.2f}s."
            )

        self.task_queue.task_done()

    except queue.Empty:

        if not self.is_running:
            break

        # TinyOS drops the CPU into ultra-low power state
        # until a hardware interrupt fires.
        print(
            "[POWER]: No pending tasks. "
            "MCU entering low-power sleep mode..."
        )
```

Hariprasad T127

```
            time.sleep(0.5)


# --- Test Execution Case ---

if __name__ == "__main__":

    simulation = TinyOSSchedulerSimulation()

    # Start the continuous TinyOS scheduler processing loop in a thread
    scheduler_thread = threading.Thread(
        target=simulation.run_scheduler_loop
    )

    scheduler_thread.start()

    # Simulate asynchronous hardware triggers
    # (e.g., sensor timers or radios)
    time.sleep(0.5)

    simulation.trigger_hardware_event(
        "Timer0.fired",
        "ReadSensorTask",
        1.5
    )

    simulation.trigger_hardware_event(
        "Radio.receive",
        "ProcessPacketTask",
        0.8
    )

    # Intentionally fire an event mid-execution
    # to observe non-preemption behavior
    time.sleep(0.4)

    simulation.trigger_hardware_event(
        "ADC.dataReady",
        "FilterDataTask",
        0.5
    )

    # Allow simulation to run through processing
    time.sleep(4.0)
```

Hariprasad T127

```
# Stop the scheduler
simulation.is_running = False

# Wait for scheduler thread to finish
scheduler_thread.join()

print("\n[STOP]: Simulation Finished")
```

Output:

